Guía QA — Ejecución en CMD de la Suite Automatizada (28 Casos)

Esta guía es para que **otra persona (QA)** ejecute por su cuenta, desde la terminal, los 28 casos de prueba automatizados documentados en `Documentation/laboratorios/Lab02_Plan_de_Pruebas_RespiCare.md` (sección XV), sin depender de que el desarrollador los corra. Es el complemento **de terminal** del `Anexo D` de ese mismo documento, que cubre las pruebas de interfaz (web/APK).

No requiere Docker ni levantar el stack completo para la mayoría de los casos: los tests de backend usan una base de datos MongoDB en memoria (`mongodb-memory-server`) que se crea y destruye sola en cada ejecución. Solo los casos de **Cypress** (web) necesitan la aplicación corriendo.

0. Prerrequisitos (una sola vez)

```
# Backend
cd backend
npm install

# AI Services
cd ../ai-services
python -m venv venv
source venv/bin/activate   # Windows: venv\Scripts\activate
pip install -r requirements-test.txt

# Web
cd ../web
npm install
npx cypress install

# Mobile
cd ../mobile
npm install
```

- **Nota:** en Windows, si `pytest` no se reconoce como comando, usar siempre `python -m pytest` en vez de `pytest` (ver `ai-services/COMANDOS_PYTEST.md`).

1. Backend (Jest) — cd backend

RF	Comando	Resultado esperado
RF-001	`npx jest tests/integration/auth.integration.test.ts`	PASS
RF-002	`npx jest tests/integration/sympto`	PASS

	`mAnalyzer.integration.test.ts`	
RF-004	`npx jest tests/unit/services/emergencyService.test.ts tests/e2e/emergency.e2e.test.ts`	PASS
RF-008	`npx jest tests/unit/models/MedicalHistory.test.ts tests/integration/medicalHistoryAdvanced.integration.test.ts`	PASS
RF-009	`npx jest tests/unit/services/alertService.test.ts tests/integration/alerts.integration.test.ts tests/e2e/alerts.e2e.test.ts`	PASS
RF-010	`npx jest tests/unit/services/automaticReportService.test.ts tests/integration/automaticReports.integration.test.ts`	PASS
RF-011	`npx jest tests/unit/services/educationalContentService.test.ts tests/integration/educationalContent.integration.test.ts`	PASS
RF-012	`npx jest tests/unit/services/healthCenterService.test.ts tests/integration/healthCenters.integration.test.ts`	PASS
RF-013	`npx jest tests/unit/services/institutionalIntegrationService.test.ts tests/integration/institutional.integration.test.ts tests/unit/middleware/institutionalAuth.test.ts`	PASS
RF-014	`npx jest tests/integration/wearables.integration.test.ts`	PASS

O todo junto:

```
npm test
```

2. AI Services (Pytest) — cd ai-services (con venv activado)

RF	Comando	Resultado esperado
RF-002	`python -m pytest tests/api/test_symptom_ml_analyzer_endpoints.py -v`	PASS
RF-003	`python -m pytest tests/api/test_audio_analyzer_endpoints.py -v`	PASS
RF-005	`python -m pytest tests/api/test_symptom_ml_analyzer_endpoints.py::TestApplyMedicalValidation -v`	PASS
RF-006	`python -m pytest tests/services/test_symptom_analysis_service.py -v`	PASS
RF-014 (vitales)	`python -m pytest tests/api/test_symptom_ml_analyzer_endpoints.py -v -k vitals`	PASS

O todo junto:

```
python -m pytest -v
```

3. Web — Cypress (requiere la app corriendo)

Cypress necesita el frontend en `http://localhost:3000`. Primero levantar el entorno (una terminal aparte):

```
bash scripts/setup-demo.sh   # Windows: .\scripts\setup-demo.ps1
```

Luego, en otra terminal, `cd web`:

RF	Comando	Resultado esperado
RF-001	`npx cypress run --spec cypress/e2e/authentication.cy.js`	PASS
RF-004	`npx cypress run --spec cypress/e2e/emergency.cy.js`	PASS
RF-007	`npx cypress run --spec cypress/e2e/doctor-validation.cy.js`	PASS
RF-008	`npx cypress run --spec cypress/e2e/medical-history.cy.js`	PASS
RF-010	`npx cypress run --spec cypress/e2e/analytics.cy.js`	PASS

O toda la suite:

```
npx cypress run
```

4. Mobile (Jest) — cd mobile

RF	Comando	Resultado esperado
RF-003	`npx jest __tests__/components/audioPermissions.test.tsx`	PASS

5. Cómo leer el resultado

- **Jest:** busca `PASS` (verde) o `FAIL` (rojo) junto al nombre del archivo, y el resumen final `Tests: X passed, X total`.
- **Pytest:** busca `passed` en la última línea (`== X passed in Ys ==`); si hay `failed`, pytest lista el nombre del test que falló.
- **Cypress:** abre un reporte en consola por spec (✓ verde = pasó, ✗ rojo = falló) y guarda video/capturas en `web/cypress/videos` y `web/cypress/screenshots` si algo falla.

6. Registro de resultados

RF	Stack	Ejecutado por	Fecha	Resultado	Observaciones
RF-001	Jest + Cypress				
RF-002	Jest + Pytest				
RF-003	Pytest + Jest (mobile)				
RF-004	Jest + Cypress				
RF-005	Pytest				
RF-006	Pytest				
RF-007	Cypress				
RF-008	Jest + Cypress				
RF-009	Jest				
RF-010	Jest + Cypress				
RF-011	Jest				
RF-012	Jest				
RF-013	Jest				
RF-014	Jest + Pytest				

- **Nota:** esta guía reproduce en terminal los mismos 28 casos de la sección XV del Lab02. Para las pruebas de interfaz (web/APK) usar el `Anexo D` de ese documento, no esta guía.